

STUDENT MANAGEMENT SYSTEM

CRUD-Based Web Application — Project Report

VSB Skill Vault — Activity 3 (Mini Web Application)

Student: Akshara KR

1. Project Overview

The Student Management System is a full-stack CRUD web application for maintaining student records. It allows users to add, view, edit, delete, and search student information through a browser-based interface. The application uses HTML, CSS and JavaScript on the frontend, Flask for the backend REST API, and SQLite for persistent storage.

2. Problem Statement

Colleges need a simple system to maintain student records such as name, roll number, email, department, year, and CGPA. This project provides a centralized web interface for creating, viewing, updating, deleting, and searching those records.

3. Objectives

- Implement complete Create, Read, Update, and Delete operations.
- Provide a responsive and user-friendly student record interface.
- Expose REST API endpoints for frontend-backend communication.
- Store records persistently using SQLite.
- Apply validation and meaningful error handling.
- Test the main CRUD operations and API behavior.

4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript (Fetch API)	User interface and client-side interaction
Backend	Python, Flask	Server-side logic and REST API
Database	SQLite	Persistent data storage
API Testing	Postman / curl	API testing
Version Control	Git / GitHub	Source-code management

5. System Architecture

Browser (HTML/CSS/JavaScript) → Fetch API → Flask REST API → SQLite Database

The browser provides the user interface. JavaScript sends HTTP requests using the Fetch API. Flask processes requests, validates data, performs database operations, and returns JSON responses. SQLite stores student records.

6. User Interface

The main interface provides an Add Student form and a Students table. The table displays roll number, name, email, department, year, CGPA, and controls for editing and deleting records. A search box is provided for finding records.

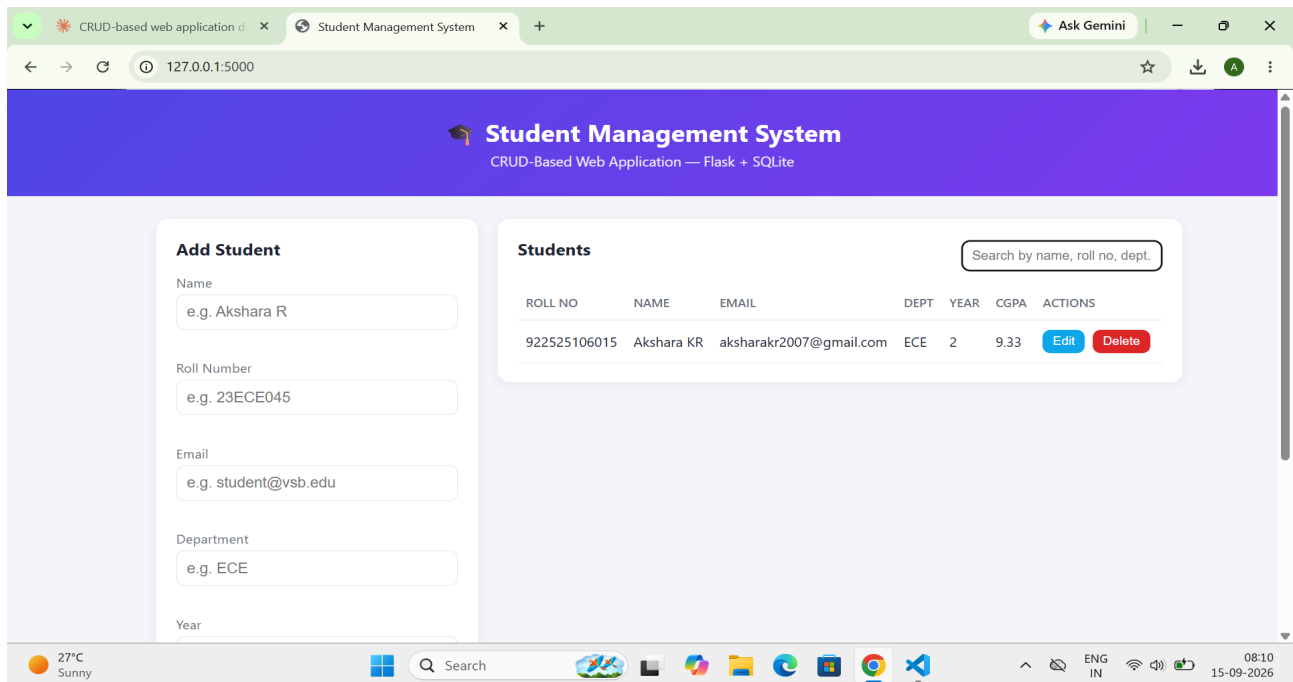


Figure 1. Student Management System — main CRUD interface.

7. Database Design

Table: students

Field	Type	Constraint
id	INTEGER	PRIMARY KEY, AUTOINCREMENT
name	TEXT	NOT NULL
roll_no	TEXT	NOT NULL, UNIQUE
email	TEXT	NOT NULL
department	TEXT	NOT NULL
year	INTEGER	NOT NULL
cgpa	REAL	—

8. REST API Endpoints

Operation	Method	Endpoint	Result
Create	POST	/api/students	New record created
Read All	GET	/api/students	List of records; supports ?search=
Read One	GET	/api/students/{id}	Single record returned
Update	PUT	/api/students/{id}	Record updated
Delete	DELETE	/api/students/{id}	Record removed

9. CRUD Functional Implementation

Create: Student details are entered through the form and submitted to the backend using POST.

Read: Student records are retrieved using GET and displayed in the table. Search is supported.

Update: Existing student information is edited and saved using PUT.

Delete: A selected student record is removed using DELETE.

10. Validation and Error Handling

- Required fields: name, roll_no, email, department, and year.
- Email format is checked using a regular expression.
- Year must be in the range 1–5 and CGPA in the range 0–10.
- Duplicate roll numbers are prevented using a database UNIQUE constraint.
- Database IntegrityError is handled and returned as a clear API error.
- Server-side validation is the authoritative validation layer.

11. Installation and Execution

```
pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000> in a browser. The SQLite database students.db is created automatically on first run.

12. Testing Results

Test Case	Expected Result	Status
Create valid data	201 – record created	Passed
Read all / read one	200 – records returned	Passed
Update valid record	200 – record updated	Passed
Missing/invalid fields	400 – field-level errors	Passed
Duplicate roll number	400 – duplicate handled	Passed
Delete valid record	200 – record removed	Passed

13. Challenges and Solutions

Duplicate roll numbers: A UNIQUE database constraint was used and IntegrityError was handled to provide a clean API error.

Consistent validation: Server-side validation was made authoritative while the UI mirrors the rules for faster feedback.

14. Project Structure

```
student-crud/
├── app.py                # Flask backend + REST API
├── requirements.txt
├── students.db           # created at runtime
├── templates/
│   └── index.html
├── static/
│   ├── css/style.css
│   └── js/app.js
```

15. GitHub Repository

<https://github.com/Aksharakr2007/student-management-crud>

The repository contains the source code, frontend assets, requirements, and project documentation.

16. Future Enhancements

- Authentication and administrator login.
- Pagination for large record sets.
- CSV/Excel export.
- Migration to MySQL or PostgreSQL for production deployment.

17. Learning Outcomes

- Understanding of full-stack web application architecture.
- Practical experience with HTML, CSS, JavaScript, Flask, REST APIs, and SQLite.
- Understanding of CRUD operations and HTTP methods.
- Application of validation, error handling, testing, Git, and GitHub.

18. Conclusion

The Student Management System demonstrates a complete CRUD-based web application with frontend, Flask REST API, SQLite database integration, validation, error handling, testing, and version control. The implementation addresses the major requirements in the supplied SOP and provides a foundation for future enhancements such as authentication, data export, pagination, and production database deployment.

Submitted by: Akshara KR